```
#Amritansh Singh 23070521013

import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST data
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize images to [-1, 1]
x_train = (x_train.astype('float32') - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

# Buffer and batch the dataset
BUFFER_SIZE = 60000
BATCH_SIZE = 256

train_dataset = tf.data.Dataset.from_tensor_slices(x_train)\
    .shuffle(BUFFER_SIZE)\
    .batch(BATCH_SIZE)

print(f"Dataset shape: {x_train.shape}")

LATENT_DIM = 100

# 1. Generator Architecture
def build_generator():
    model = models.Sequential(name="Generator")

    # Foundation for 7x7 image
    model.add(layers.Input(shape=(LATENT_DIM,)))
    model.add(layers.Dense(7 * 7 * 128))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Reshape((7, 7, 128)))

    # Upsample to 14x14
    model.add(layers.Conv2DTranspose(64, kernel_size=4, strides=2, padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(negative_slope=0.2))

    # Upsample to 28x28
    model.add(layers.Conv2DTranspose(1, kernel_size=4, strides=2, padding='same', activation='tanh
    return model

# 2. Discriminator Architecture
def build_discriminator():
    model = models.Sequential(name="Discriminator")

    model.add(layers.Input(shape=(28, 28, 1)))
    model.add(layers.Conv2D(64, kernel_size=4, strides=2, padding='same'))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, kernel_size=4, strides=2, padding='same'))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

generator = build_generator()
discriminator = build_discriminator()
```

```
generator.summary()
discriminator.summary()

# Optimizers
generator_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)

# Fixed: BinaryCrossentropy with lowercase 'entropy'
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables)
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainabl

    return gen_loss, disc_loss

EPOCHS = 7
seed = tf.random.normal([16, LATENT_DIM])

print("Training GAN...")
for epoch in range(EPOCHS):
    gen_loss_list = []
    disc_loss_list = []

    for image_batch in train_dataset:
        if len(image_batch) == BATCH_SIZE:
            g_loss, d_loss = train_step(image_batch)
            gen_loss_list.append(g_loss)
            disc_loss_list.append(d_loss)

    print(f"Epoch {epoch + 1}/{EPOCHS} | Gen Loss: {np.mean(gen_loss_list):.4f} | Disc Loss: {np.m

print("Training Complete!")

# Generate images from fixed seed
predictions = generator(seed, training=False)

# Rescale from [-1, 1] back to [0, 1] for visualization
predictions = (predictions + 1.0) / 2.0

plt.figure(figsize=(6, 6))
for i in range(predictions.shape[0]):
    plt.subplot(4, 4, i + 1)
    plt.imshow(predictions[i, :, :, 0], cmap='gray')
```

```
    plt.axis('off')

plt.suptitle("Generated Synthetic MNIST Digits", fontsize=14)
plt.tight_layout()
plt.show()
```

```
Dataset shape: (60000, 28, 28, 1)
```

Model: "Generator"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 6272)	633,472
batch_normalization_2 (BatchNormalization)	(None, 6272)	25,088
leaky_re_lu_4 (LeakyReLU)	(None, 6272)	0
reshape_1 (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 14, 14, 64)	131,136
batch_normalization_3 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_5 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_3 (Conv2DTranspose)	(None, 28, 28, 1)	1,025

Start coding or generate with AI.

Non-trainable params: 12,672 (49.50 KB)

Model: "Discriminator"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 14, 14, 64)	1,088
leaky_re_lu_6 (LeakyReLU)	(None, 14, 14, 64)	0
dropout_2 (Dropout)	(None, 14, 14, 64)	0
conv2d_3 (Conv2D)	(None, 7, 7, 128)	131,200
leaky_re_lu_7 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_3 (Dropout)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_3 (Dense)	(None, 1)	6,273

Total params: 138,561 (541.25 KB)
Trainable params: 138,561 (541.25 KB)
Non-trainable params: 0 (0.00 B)

Training GAN